

seRef Hook

Definizione. `useRef` crea un contenitore mutabile che **persiste tra i render** ma il cui cambiamento **non provoca un nuovo render**. Ha due usi distinti.

```
const riferimento = useRef(valoreIniziale);  
  
// il valore si legge e si scrive con riferimento.current
```

Uso 1 — accedere direttamente a un elemento del DOM. È il caso principale.

```
const Form = () => {  
  const inputRef = useRef(null);  
  
  useEffect(() => {  
    inputRef.current.focus();    // focus automatico al caricamento  
  }, []);  
  
  const gestisciSubmit = (e) => {  
    e.preventDefault();  
    console.log(inputRef.current.value); // leggo il valore senza stato  
  };  
  
  return (  
    <form onSubmit={gestisciSubmit}>  
      <input type="text" ref={inputRef} />  
      <button type="submit">Invia</button>  
    </form>  
  );  
};
```

L'attributo `ref={inputRef}` collega il riferimento all'elemento reale. Da quel momento `inputRef.current` è il nodo DOM, con tutti i suoi metodi: `.focus()`, `.value`, `.scrollIntoView()`, `.select()`.

Il valore iniziale è `null` perché al primo render l'elemento non esiste ancora: viene popolato subito dopo. Per questo l'accesso va fatto dentro `useEffect` o in un gestore di eventi, mai durante il render.

Uso 2 — memorizzare un valore senza causare render.

```
const Componente = () => {
```

```

const [valore, setValore] = useState(0);

const primoRender = useRef(true);

useEffect(() => {
  if (primoRender.current) {
    primoRender.current = false;
    return;           // salta l'effetto al primo giro
  }

  console.log("Effetto dai render successivi");
}, [valore]);
};

```

Questo pattern serve a ignorare il montaggio iniziale — utile quando l'effetto deve reagire solo ai *cambiamenti* di un valore, non alla sua comparsa.

La differenza con `useState`, che è il punto centrale:

	<code>useState</code>	<code>useRef</code>
Sopravvive ai render	Sì	Sì
Causa un nuovo render	Sì	No
Sintassi di lettura	<code>valore</code>	<code>referimento.current</code>
Modifica	tramite il setter	diretta
Quando usarlo	il valore si vede a schermo	il valore non si vede, o serve il DOM
La regola pratica: <i>se cambiandolo lo schermo deve aggiornarsi, è stato; altrimenti è un ref.</i>		
Contatori nascosti, id di timer, valori del render precedente, riferimenti a nodi DOM: tutti <code>useRef</code> .		

CONCETTI AVANZATI E REACT HOOKS (Parte 2)

`useReducer` — set up base e concetti

3.22 `useReducer` — set up base e concetti

Definizione. `useReducer` è un hook alternativo a `useState` per gestire **stato complesso**: quando hai più valori correlati fra loro e più azioni diverse che li modificano. Centralizza tutta la logica di aggiornamento in un'unica funzione, il *reducer*.

Il problema che risolve. Il corso lo costruisce nel modo giusto: prima realizza un'app con soli `useState`, poi fa il refactoring. Immagina un modal con una lista:

```
const [persone, setPersone] = useState([]);
const [modalAperto, setModalAperto] = useState(false);
const [messaggioModal, setMessaggioModal] = useState("");
const [nome, setNome] = useState("");
```

Ogni azione dell'utente deve toccare più stati insieme. "Aggiungi persona" fa quattro cose: aggiunge alla lista, apre il modal, imposta il messaggio, svuota l'input. "Rimuovi persona" ne fa tre. La logica si sparpaglia in funzioni diverse, e diventa difficile capire *tutti* i modi in cui lo stato può cambiare.

I tre concetti di useReducer:

- **State:** l'oggetto che contiene tutto lo stato.
- **Action:** un oggetto che descrive *cosa è successo*. Ha sempre un `type` (stringa) e spesso un `payload` (i dati).
- **Reducer:** una funzione pura (`state, action`) => `nuovoState`. Riceve lo stato attuale e l'azione, e ritorna il nuovo stato.

Il nome "reducer" viene da `Array.reduce`: anche lì una funzione riceve un accumulatore e un elemento, e ritorna un nuovo accumulatore.

```
import { useReducer } from 'react';

const statoIniziale = {
  persone: [],
  modalAperto: false,
  messaggioModal: ""
};

const reducer = (state, action) => {
  if (action.type === 'AGGIUNGI_PERSONA') {
    const nuovePersone = [...state.persone, action.payload];
    return {
      ...state,
      persone: nuovePersone,
      modalAperto: true,
      messaggioModal: "Persona aggiunta"
    };
  }

  if (action.type === 'NESSUN_VALORE') {
```

```

    return { ...state, modalAperto: true, messaggioModal: "Inserisci un nome" };
  }
  if (action.type === 'CHIUDI_MODAL') {
    return { ...state, modalAperto: false };
  }
  if (action.type === 'RIMUOVI_PERSONA') {
    const nuovePersone = state.persone.filter((p) => p.id !== action.payload);
    return { ...state, persone: nuovePersone };
  }
  throw new Error(`Nessuna azione corrispondente a: ${action.type}`);
};

```

Le regole del reducer, tutte importanti:

1. **Deve sempre ritornare uno stato**, mai undefined. Il throw finale serve proprio a intercettare i typo nei nomi delle azioni.
2. **Deve essere una funzione pura**: niente chiamate API, niente `Math.random()`, niente modifiche a variabili esterne. Solo calcolo.
3. **Deve ritornare un oggetto nuovo**, mai modificare `state`. Da qui lo `...state` all'inizio di ogni ritorno.
4. Sta **fuori dal componente**, perché non dipende da esso.

La forma più diffusa usa `switch` invece degli `if`:

```

const reducer = (state, action) => {
  switch (action.type) {
    case 'AGGIUNGI_PERSONA':
      return { ...state, persone: [...state.persone, action.payload] };
    case 'CHIUDI_MODAL':
      return { ...state, modalAperto: false };
    default:
      throw new Error(`Azione non riconosciuta: ${action.type}`);
  }
};

```

useReducer — dispatch function

Definizione. `dispatch` è la funzione che "spedisce" un'azione al reducer. È l'unico modo per far cambiare lo stato.

```
const Componente = () => {

  const [state, dispatch] = useReducer(reducer, statoIniziale);

  const [nome, setNome] = useState("");

  const gestisciSubmit = (e) => {

    e.preventDefault();

    if (nome) {

      const nuovaPersona = { id: Date.now(), nome };

      dispatch({ type: 'AGGIUNGI_PERSONA', payload: nuovaPersona });

      setNome("");

    } else {

      dispatch({ type: 'NESSUN_VALORE' });

    }

  };

  return (

    <>

      {state.modalAperto && (

        <Modal messaggio={state.messaggioModal} chiudi={() => dispatch({ type:

'CHIUDI_MODAL' })} />

      )}

      <form onSubmit={gestisciSubmit}>

        <input value={nome} onChange={(e) => setNome(e.target.value)} />

        <button type="submit">Aggiungi</button>

      </form>

      {state.persone.map(({ id, nome }) => (

        <div key={id}>

          <h4>{nome}</h4>

          <button onClick={() => dispatch({ type: 'RIMUOVI_PERSONA', payload: id })}>
```

```

    Rimuovi
  </button>
</div>
)}}
</>
);
};

```

La firma è simmetrica a `useState`, e questo aiuta a ricordarla:

```

const [state, dispatch] = useReducer(reducer, statoIniziale);
//   ↑      ↑         ↑      ↑
// stato  spedisce  la funzione  valore
//      azioni    pura      iniziale

```

useReducer — payload e action

Definizione. Il payload è il carico di dati che accompagna l'azione. La convenzione `{ type, payload }` viene dal mondo Redux ed è lo standard di fatto.

```

dispatch({ type: 'AGGIUNGI', payload: { id: 1, nome: "Anna" } });
dispatch({ type: 'RIMUOVI', payload: 3 });
dispatch({ type: 'CHIUDI_MODAL' }); // senza payload, se non serve

```

Convenzioni utili:

- `type` in MAIUSCOLO con underscore, per distinguerlo a colpo d'occhio dal resto del codice.
- Definire le action type come costanti in un file separato, per evitare typo:

```

// actions.js export const AGGIUNGI_PERSONA = 'AGGIUNGI_PERSONA';

```

Quando scegliere useReducer al posto di useState, in sintesi:

Situazione	Scelta
1-2 valori indipendenti	<code>useState</code>
Stato con più campi correlati	<code>useReducer</code>
Il nuovo stato dipende molto dal precedente	<code>useReducer</code>
Molte azioni diverse sullo stesso stato	<code>useReducer</code>
Vuoi testare la logica separatamente	<code>useReducer</code> (il reducer è una funzione pura, testabile da sola)

E soprattutto: **useReducer è il ponte concettuale verso Redux** della Lezione 5. Chi capisce reducer, action e dispatch qui, in Redux si ritrova in casa.

Passaggio di proprietà a cascata (Prop Drilling)

Definizione. Il *prop drilling* è il passaggio di props attraverso componenti intermedi che non ne hanno bisogno, solo per farle arrivare a un componente profondo nell'albero.

La trascrizione lo definisce bene: *"il passaggio a cascata di proprietà e funzioni da un componente padre a un componente figlio"*.

```
const App = () => {  
  const [persone, setPersone] = useState(data);  
  const rimuovi = (id) => setPersone(persone.filter((p) => p.id !== id));  
  
  return <Elenco persone={persone} rimuovi={rimuovi} />;  
};  
  
// Elenco non usa "rimuovi": la passa soltanto  
const Elenco = ({ persone, rimuovi }) => (  
  <div>  
    {persone.map((p) => <Persona key={p.id} {...p} rimuovi={rimuovi} />)}  
  </div>  
);  
  
// Solo qui viene finalmente usata  
const Persona = ({ id, nome, rimuovi }) => (  
  <article>  
    <h5>{nome}</h5>  
    <button onClick={() => rimuovi(id)}>X</button>  
  </article>  
);
```

I problemi. Con due livelli è gestibile. Con cinque o sei diventa un peso: ogni componente intermedio va modificato per aggiungere una prop che non lo riguarda, il codice si riempie di props "di passaggio", e rinominare qualcosa significa toccare l'intera catena.

Importante: il prop drilling **non è sempre un errore**. Per uno o due livelli è la soluzione più semplice e leggibile. Diventa un problema quando la catena si allunga o quando il dato serve a molti rami dell'albero.

useContext Hook

Definizione. Il **Context** è un meccanismo per rendere un valore disponibile a tutti i componenti di un sottoalbero, **senza passarlo esplicitamente di livello in livello**. `useContext` è l'hook che lo legge.

Tre pezzi:

1. Creare il contesto

```
import { createContext, useContext, useState } from 'react';
```

```
const AppContext = createContext();
```

2. Il Provider — fornisce il valore

```
const AppProvider = ({ children }) => {  
  const [persone, setPersone] = useState(data);  
  const rimuovi = (id) => setPersone(persone.filter((p) => p.id !== id));  
  
  return (  
    <AppContext.Provider value={{ persone, rimuovi }}>  
      {children}  
    </AppContext.Provider>  
  );  
};
```

Nota l'uso di `children`, il concetto della Lezione 2: il Provider avvolge l'albero e lo lascia passare intatto.

3. Il consumo — nel componente che serve

```
const Persona = ({ id, nome }) => {  
  const { rimuovi } = useContext(AppContext);  
  return (  
    <article>  
      <h5>{nome}</h5>  
      <button onClick={() => rimuovi(id)}>X</button>  
    </article>  
  );  
};
```

```
};
```

E l'app si avvolge così, di solito in `index.js`:

```
<AppProvider>
  <App />
</AppProvider>
```

E lenco adesso non passa più nulla: la catena è saltata completamente.

Il pattern del custom hook per il contesto. Il corso lo usa in tutti i progetti (nella trascrizione è `useGlobalContext`):

```
export const useGlobalContext = () => {
  const context = useContext(AppContext);
  if (context === undefined) {
    throw new Error("useGlobalContext deve essere usato dentro AppProvider");
  }
  return context;
};
```

Due vantaggi: non devi importare `AppContext` in ogni file, e ottieni un errore chiaro se usi il contesto fuori dal Provider — invece di un `undefined` misterioso.

Nel componente diventa una riga sola:

```
const { persone, rimuovi } = useGlobalContext();
```

Avvertenza sulle performance. Quando il valore del Provider cambia, **tutti** i componenti che consumano quel contesto vengono ridisegnati. Per questo non si mette tutto in un unico contesto gigante: meglio contesti separati per dominio (tema, utente, carrello), o valori memorizzati con `useMemo`.

Quando usare Context. Per dati davvero globali: tema chiaro/scuro, utente autenticato, lingua, carrello, apertura della sidebar. Non è un sostituto universale delle props: per dati locali, le props restano più semplici e più esplicite.

Il pattern completo che il corso usa nei progetti 8, 9 e 10 è `useContext` + `useReducer`: il reducer gestisce lo stato complesso, il context lo distribuisce. È letteralmente una versione artigianale di Redux.

Custom Hook — creare hook personalizzati

Definizione. Un custom hook è **una normale funzione JavaScript che inizia con `use` e che al suo interno usa altri hook**. Serve a riutilizzare **logica**, non interfaccia.

La trascrizione centra il punto: React permette di riutilizzare i componenti, cioè gli elementi visivi; i custom hook permettono di riutilizzare *un aspetto funzionale*.

Esempio — useFetch, quello sviluppato nel corso. Partiamo dal codice duplicato:

```
// Componente Utenti

const [dati, setDati] = useState([]);
const [caricamento, setCaricamento] = useState(true);
useEffect(() => { /* fetch */ }, []);

// Componente Post — identico, solo con un altro URL

const [dati, setDati] = useState([]);
const [caricamento, setCaricamento] = useState(true);
useEffect(() => { /* fetch */ }, []);
```

Estraiamo:

```
// useFetch.js

import { useState, useEffect } from 'react';

const useFetch = (url) => {
  const [dati, setDati] = useState([]);
  const [caricamento, setCaricamento] = useState(true);
  const [errore, setErrore] = useState(null);

  useEffect(() => {
    const recupera = async () => {
      setCaricamento(true);

      try {
        const risposta = await fetch(url);
        if (!risposta.ok) throw new Error("Richiesta fallita");
        const risultato = await risposta.json();
        setDati(risultato);
        setErrore(null);
      } catch (err) {
        setErrore(err.message);
      }
    };
  });
}
```

```

    } finally {
      setCaricamento(false);
    }
  };
  recupera();
}, [url]);    // rigira se cambia l'URL

return { dati, caricamento, errore };
};

export default useFetch;

```

E l'uso:

```

const Utenti = () => {
  const { dati, caricamento, errore } = useFetch("https://api.github.com/users");

  if (caricamento) return <h2>Caricamento...</h2>;
  if (errore) return <h2>{errore}</h2>;
  return <ul>{dati.map((u) => <li key={u.id}>{u.login}</li>)}</ul>;
};

const Post = () => {
  const { dati, caricamento } = useFetch("https://api.esempio.com/posts");
  // ...
};

```

Due componenti, due righe, zero duplicazione.

Le regole dei custom hook:

1. **Il nome deve iniziare con use.** Non è estetica: è il segnale che dice a React e al linter di applicare le regole degli hook a quella funzione.
2. **Può chiamare altri hook** — anzi, è l'unico tipo di funzione che può farlo oltre ai componenti.
3. **Ritorna quello che vuoi:** un oggetto, un array, un singolo valore.
4. **Ogni chiamata è indipendente.** Due componenti che usano `useFetch` hanno stati completamente separati. Non si condivide nulla — quello è il compito del Context.

Altri custom hook classici, che ritroverai nei progetti:

```
useToggle()      // gestisce un booleano on/off
useLocalStorage() // stato sincronizzato con localStorage
useTitle()       // aggiorna il titolo della pagina (progetto Cocktail)
useWindowSize()  // larghezza e altezza della finestra
useDebounce()    // ritarda un valore, utile nelle ricerche
```

PropTypes — concetti base

Definizione. Le PropTypes sono un sistema di **validazione a runtime** dei tipi delle props. Durante lo sviluppo avvisano in console se un componente riceve props sbagliate o mancanti.

Il problema reale, ben descritto nella trascrizione: quando fai fetch da un'API, può capitare che un oggetto dell'array **non abbia tutte le proprietà** degli altri. Se il componente prova ad accedere a `image.url` e `image` non esiste, l'app va in crash con `Cannot read property 'url' of undefined` — una schermata bianca.

```
npm install prop-types
```

```
import PropTypes from 'prop-types';
```

```
const Prodotto = ({ nome, prezzo, image }) => (
  <article className="card">
    <img src={image.url} alt={nome} />
    <h4>{nome}</h4>
    <p>€ {prezzo}</p>
  </article>
);
```

```
Prodotto.propTypes = {
  nome: PropTypes.string.isRequired,
  prezzo: PropTypes.number.isRequired,
  image: PropTypes.object.isRequired
};
```

```
export default Prodotto;
```

Ora, se `prezzo` arriva come stringa o manca del tutto, in console appare un avviso chiaro che ti dice **quale componente** e **quale prop**.

I validatori principali:

```
PropTypes.string
PropTypes.number
PropTypes.bool
PropTypes.func          // per le funzioni passate come prop
PropTypes.array
PropTypes.object
PropTypes.node          // qualsiasi cosa renderizzabile (usato per children)
PropTypes.arrayOf(PropTypes.number)
PropTypes.shape({ id: PropTypes.number, nome: PropTypes.string })
PropTypes.oneOf(['piccolo', 'medio', 'grande'])
.isRequired si aggiunge in coda per rendere obbligatoria la prop.
```

PropTypes — valori di default

Definizione. I *default* sono valori di ripiego usati quando una prop non viene passata. Evitano il crash invece di limitarsi ad avvisare.

Metodo classico:

```
Prodotto.defaultProps = {
  nome: "Nome non disponibile",
  prezzo: 0,
  image: { url: "https://via.placeholder.com/150" }
};
```

Metodo moderno, preferibile — default direttamente nella destrutturazione:

```
const Prodotto = ({
  nome = "Nome non disponibile",
  prezzo = 0,
  image = { url: placeholder }
}) => { ... };
```

È JavaScript puro, funziona senza librerie, ed è il metodo consigliato oggi (defaultProps sulle function component è in via di deprecazione).

Attenzione: il default scatta solo se la prop è `undefined`. Se arriva `null`, il default **non** viene applicato e il crash resta.

Nota sul contesto attuale. Le PropTypes sono state ampiamente sostituite da **TypeScript**, che fa controlli a *compile time* (prima di eseguire) invece che a runtime, ed è oggi lo standard nei progetti professionali. Capire le PropTypes resta utile per leggere codice esistente e per afferrare il concetto di contratto fra componenti — che in TypeScript è lo stesso, scritto meglio.

React Router — introduzione

Definizione. React Router è la libreria che gestisce la **navigazione** in una Single Page Application: mostra componenti diversi in base all'URL, senza ricaricare la pagina.

Il problema, spiegato bene nella trascrizione. Esiste un solo `index.html`. Se scrivi `/about` nella barra degli indirizzi, non esiste nessun file `about.html` da servire. Serve qualcosa che intercetti l'URL e decida quale componente iniettare nel `div root`.

"Non possiamo creare altre pagine, ma dobbiamo fare un render differente dei componenti a seconda del path sul quale l'utente si trova."

v5 o v6? Il corso copre entrambe, con due sezioni separate. Come dice l'autore, sui progetti reali potresti trovare una versione vecchia mantenuta per stabilità. Ma per imparare oggi, **usa la v6**: la v5 ha una sintassi diversa (`Switch`, `component={}`, `useHistory`) che non ti serve se non devi mettere mano a codice legacy. Qui tratto la v6.

```
npm install react-router-dom
```

v6 — BrowserRouter, Routes & Route

Definizione. I tre componenti fondamentali:

- **BrowserRouter** (rinominato per convenzione in `Router`): avvolge l'applicazione e abilita tutto il sistema di routing.
- **Routes**: il contenitore delle rotte. Sceglie **la sola rotta che corrisponde meglio** all'URL corrente. Ha sostituito lo `Switch` della v5.
- **Route**: la singola associazione fra un percorso e un componente.

```
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';  
  
import Home from './screens/Home';  
  
import About from './screens/About';  
  
import Profile from './screens/Profile';  
  
import ErrorPage from './screens/ErrorPage';
```

```
const App = () => {  
  return (  
    <Router>
```

```

<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
  <Route path="/profile" element={<Profile />} />
  <Route path="*" element={<ErrorPage />} />
</Routes>

</Router>

);
};

```

Convenzione di cartelle, presa dalla trascrizione: i componenti-pagina vanno in una cartella `screens/` o `pages/`, distinta da `components/`. È una separazione che si mantiene in tutti i progetti seri.

Differenze v5 → v6 da tenere a mente:

v5	v6
<code><Switch></code>	<code><Routes></code>
<code>component={Home}</code>	<code>element={<Home />}</code>
<code>exact path="/"</code>	non serve più: il match è esatto per default
<code>useHistory()</code>	<code>useNavigate()</code>
Rotte annidate manuali	<code><Outlet /></code>
Il <code>path="*" in fondo</code> è la rotta catch-all: intercetta qualsiasi URL non riconosciuto e mostra la pagina 404.	

v6 — Link ed ErrorPage

Definizione. `Link` è il componente che sostituisce il tag `<a>`. Cambia l'URL **senza ricaricare la pagina**.

```

import { Link } from 'react-router-dom';

const Navbar = () => (
  <nav>
    <Link to="/">Home</Link>
    <Link to="/about">About</Link>
    <Link to="/profile">Profilo</Link>
  </nav>
);

```

Perché non usare `<a href="/about">`. Il tag `<a>` provoca una richiesta HTTP al server e un ricaricamento completo: perdi tutto lo stato dell'applicazione e annulli il vantaggio della SPA. `Link` intercetta il click, chiama `preventDefault`, aggiorna l'URL via History API e lascia che React Router ridisegni solo ciò che serve.

NavLink è la variante che sa se è attiva — utile per evidenziare la voce di menu corrente:

```
<NavLink to="/about" className={({ isActive }) => isActive ? "attivo" : ""}>
  About
</NavLink>
```

La navbar va messa **dentro il Router ma fuori da Routes**, così resta visibile su tutte le pagine:

```
<Router>
  <Navbar />
  <Routes>...</Routes>
  <Footer />
</Router>
```

v6 — Template, useParams e useNavigate

Definizione. I **parametri dinamici** permettono a una sola rotta di servire infinite pagine: `/persona/1`, `/persona/2`, e così via.

Dichiarazione con i due punti:

```
<Route path="/persona/:id" element={<SingolaPersona />} />
```

`useParams` — leggere il parametro:

```
import { useParams } from 'react-router-dom';

const SingolaPersona = () => {
  const { id } = useParams();
  const persona = data.find((p) => p.id === parseInt(id));

  if (!persona) return <h2>Persona non trovata</h2>;
  return <h2>{persona.nome}</h2>;
};
```

Attenzione: **il parametro è sempre una stringa**, anche se l'URL contiene un numero. Da qui il `parseInt` (o il confronto con `==` invece di `===`). È un errore molto comune.

Il link dinamico si costruisce così:

```
{persone.map((p) => (  
  <Link key={p.id} to={`/${p.id}`}>{p.nome}</Link>  
))}
```

useNavigate — **navigare da codice**, cioè senza che l'utente clicchi un link:

```
import { useNavigate } from 'react-router-dom';  
  
const Login = () => {  
  const navigate = useNavigate();  
  
  const gestisciLogin = async () => {  
    await autentica();  
    navigate('/dashboard'); // reindirizza dopo il login  
  };  
  
  return (  
    <>  
      <button onClick={gestisciLogin}>Accedi</button>  
      <button onClick={() => navigate(-1)}>Indietro</button>  
    </>  
  );  
};
```

`navigate(-1)` equivale al tasto indietro del browser. `navigate('/percorso', { replace: true })` sostituisce la voce nella cronologia invece di aggiungerla — utile dopo un login, per impedire che il tasto indietro riporti alla pagina di accesso.

v6 — Outlet e Nested Routes

Definizione. Le **rotte annidate** permettono a un layout comune di restare fisso mentre cambia solo una parte della pagina. `Outlet` è il segnaposto in cui viene inserita la rotta figlia.

```
<Routes>  
  <Route path="/" element={<SharedLayout />}>  
    <Route index element={<Home />} />  
  </Route>  
</Routes>
```

```

<Route path="about" element={<About />} />
<Route path="prodotti" element={<Prodotti />} />
<Route path="prodotti/:id" element={<SingoloProdotto />} />
<Route path="*" element={<ErrorPage />} />
</Route>
</Routes>

```

```

import { Outlet } from 'react-router-dom';

const SharedLayout = () => (
  <>
    <Navbar />
    <Outlet />    {/* qui viene inserita la rotta figlia */}
    <Footer />
  </>
);

```

Due dettagli:

- I percorsi figli **non hanno lo slash iniziale**: sono relativi al genitore.
- **index** identifica la rotta predefinita del genitore, quella mostrata su /.

Navbar e Footer vengono montati una volta sola e non si ridisegnano cambiando pagina. È esattamente la struttura usata nel progetto Cocktail.

v6 — useRoutes

Definizione. `useRoutes` permette di dichiarare le rotte come **array di oggetti JavaScript** invece che come JSX.

```

import { useRoutes } from 'react-router-dom';

const App = () => {
  const elemento = useRoutes([
    {
      path: '/',
      element: <SharedLayout />,
    },
  ]);

```

```

children: [
  { index: true, element: <Home /> },
  { path: 'about', element: <About /> },
  { path: '*', element: <ErrorPage /> }
]
}
]);

return elemento;
};

```

Quando serve. Quando le rotte vanno generate dinamicamente: per esempio filtrate in base ai permessi dell'utente, o caricate da una configurazione. Essendo un array, puoi manipolarlo con `filter` e `map` — cosa impossibile con il JSX.

Ottimizzazione — React.memo e useMemo

Qui si entra nella parte sulle performance. Due nomi quasi identici e due funzioni completamente diverse: la trascrizione dice che sono *"spesso fonte di confusione"* e che hanno *"un nome fatto apposta per confondere"*. Vero.

Il problema di partenza. Una lista di 40 profili e un contatore sopra. Ogni click sul contatore fa ridisegnare **tutti e 40** gli elementi della lista, anche se non sono cambiati. Lo verifichi mettendo un `console.log` dentro l'item: vedi 40 log ad ogni click.

Perché succede. Quando un componente si ridisegna, **tutti i suoi figli si ridisegnano**, a prescindere dal fatto che le loro props siano cambiate.

React.memo

Definizione. `React.memo` è una funzione di ordine superiore che avvolge un componente e **blocca il suo render se le props non sono cambiate**. Restituisce il risultato precedente.

```

import React from 'react';

const Item = ({ nome, immagine }) => {
  console.log("render di", nome);
  return (
    <div>
      <img src={immagine} alt={nome} />
      <h4>{nome}</h4>
    </div>
  );
};

```

```
</div>
```

```
);
```

```
};
```

```
export default React.memo(Item);
```

Ora il click sul contatore non ridisegna più la lista. Il `console.log` sparisce.

Il confronto è superficiale (`shallow`): props primitive vengono confrontate per valore, oggetti, array e funzioni **per riferimento**. È qui che nascono i problemi che i due hook successivi risolvono.

`useMemo`

Definizione. `useMemo` **memorizza il risultato di un calcolo** e lo ricalcola solo quando cambiano le sue dipendenze.

```
const Componente = ({ prodotti }) => {
```

```
  const [contatore, setContatore] = useState(0);
```

```
  const piuCostoso = useMemo(() => {
```

```
    console.log("calcolo in corso...");
```

```
    return prodotti.reduce((max, p) => (p.prezzo > max.prezzo ? p : max), prodotti[0]);
```

```
  }, [prodotti]);
```

```
  return (
```

```
    <>
```

```
    <h2>Il più costoso: {piuCostoso.nome}</h2>
```

```
    <button onClick={() => setContatore(contatore + 1)}>{contatore}</button>
```

```
  </>
```

```
);
```

```
};
```

Senza `useMemo`, il calcolo verrebbe rifatto ad ogni click del contatore, pur non c'entrando nulla. Con `useMemo`, viene rifatto solo quando cambia `prodotti`.

La differenza in una riga:

- `React.memo` → memorizza **un componente** (evita render inutili)
 - `useMemo` → memorizza **un valore** (evita calcoli inutili)
-

Ottimizzazione — useCallback

Definizione. `useCallback` memorizza **una funzione**, in modo che mantenga lo stesso riferimento tra un render e l'altro.

Perché serve. È il complemento indispensabile di `React.memo`. Guarda questo caso:

```
const App = () => {  
  const [contatore, setContatore] = useState(0);  
  const [prodotti, setProdotti] = useState(data);  
  
  const rimuovi = (id) => {  
    setProdotti(prodotti.filter((p) => p.id !== id));  
  };  
  
  return (  
    <>  
      <button onClick={() => setContatore(contatore + 1)}>{contatore}</button>  
      <Lista prodotti={prodotti} rimuovi={rimuovi} />  
    </>  
  );  
};
```

Anche se `Lista` è avvolta in `React.memo`, **continua a ridisegnarsi ad ogni click**. Il motivo: `rimuovi` è una funzione dichiarata nel corpo del componente, quindi ad ogni render viene creata **una funzione nuova**, con un riferimento diverso. Il confronto superficiale di `memo` vede una prop cambiata e ridisegna.

La soluzione:

```
const rimuovi = useCallback((id) => {  
  setProdotti((prev) => prev.filter((p) => p.id !== id));  
}, []);
```

Ora la funzione mantiene lo stesso riferimento e `React.memo` fa il suo lavoro. Nota l'uso della forma funzionale del setter: permette di lasciare l'array di dipendenze vuoto, perché non serve leggere `prodotti` dall'esterno.

La trappola dell'oggetto nelle dipendenze di `useEffect`, che la trascrizione tocca: se passi un oggetto o una funzione come dipendenza di `useEffect`, l'effetto gira ad ogni render per lo stesso motivo. `useCallback` e `useMemo` sono la cura.

Riassunto del trio:

Strumento	Memorizza	Problema che risolve
React.memo	un componente	render inutili di un figlio
useMemo	un valore	calcoli costosi ripetuti
useCallback	una funzione	riferimenti nuovi che rompono memo e useEffect

Avvertenza importante, che vale più di tutto il resto. Queste tre ottimizzazioni **hanno un costo**: React deve conservare i valori vecchi e fare confronti. Applicarle ovunque per abitudine rende l'app più lenta e il codice più difficile da leggere. La regola è: **prima misura, poi ottimizza**. Usa React DevTools Profiler per trovare il collo di bottiglia reale, poi intervieni solo lì.

Ottimizzazione — useTransition

Definizione. `useTransition` (React 18) permette di marcare un aggiornamento di stato come **non urgente**, così che non blocchi l'interfaccia.

Il problema. Un campo di ricerca che filtra una lista di 20.000 elementi. Ogni carattere digitato provoca un ricalcolo pesante, e l'input diventa scattoso: digiti "abc" e i caratteri compaiono con ritardo visibile.

```
import { useState, useTransition } from 'react';

const Ricerca = () => {
  const [testo, setTesto] = useState('');
  const [lista, setLista] = useState([]);
  const [isPending, startTransition] = useTransition();

  const gestisciCambio = (e) => {
    setTesto(e.target.value); // URGENTE: l'input risponde subito

    startTransition(() => {
      const nuova = [];
      for (let i = 0; i < 20000; i++) {
        nuova.push(e.target.value);
      }
      setLista(nuova); // NON urgente: può attendere
    });
  };
};
```

```

return (
  <>
    <input value={testo} onChange={gestisciCambio} />
    {isPending
      ? <h2>Caricamento...</h2>
      : lista.map((item, i) => <p key={i}>{item}</p>)}
    </>
  );
};

```

Come funziona. React dà priorità all'aggiornamento dell'input; il ricalcolo della lista viene interrotto e riprogrammato se l'utente continua a digitare. `isPending` è un booleano che ti permette di mostrare un indicatore nel frattempo.

Si chiamano **concurrent features**: React può ora interrompere e riprendere il lavoro di rendering, invece di eseguirlo tutto d'un fiato.

Ottimizzazione — `useDeferredValue`

Definizione. `useDeferredValue` fa la stessa cosa di `useTransition`, ma agisce **sul valore** invece che sull'aggiornamento.

```

import { useState, useDeferredValue, memo } from 'react';

const App = () => {
  const [testo, setTesto] = useState('');
  const testoRitardato = useDeferredValue(testo);

  return (
    <>
      <input value={testo} onChange={(e) => setTesto(e.target.value)} />
      <ListaPesante testo={testoRitardato} />
    </>
  );
};

```

```
const ListaPesante = memo(({ testo }) => {  
  const elementi = [];  
  for (let i = 0; i < 20000; i++) elementi.push(<p key={i}>{testo}</p>);  
  return <>{elementi}</>;  
});
```

`testo` si aggiorna subito (l'input è reattivo), `testoRitardato` "insegue" con un ritardo gestito da React.

Quando usare l'uno o l'altro:

- **useTransition** → quando controlli tu il codice che aggiorna lo stato.
- **useDeferredValue** → quando ricevi un valore dall'esterno (una prop, un hook di libreria) e non puoi intervenire sull'aggiornamento.

Va quasi sempre abbinato a `memo` sul componente pesante, altrimenti il vantaggio si perde.

useId Hook

Definizione. `useId` genera un identificativo univoco e **stabile** per collegare elementi di un form fra loro.

```
import { useId } from 'react';  
  
const CampoEmail = () => {  
  const id = useId();  
  return (  
    <>  
      <label htmlFor={id}>Email</label>  
      <input id={id} type="email" />  
    </>  
  );  
};
```

Perché non usare `Math.random()`. Perché genererebbe un id diverso ad ogni render, e soprattutto un id diverso sul server e sul client — cosa che rompe il *server-side rendering* con un errore di *hydration mismatch*. `useId` garantisce coerenza fra i due.

Cosa NON è. Non serve per le key delle liste. Quelle devono derivare dai dati (`item.id`), non essere generate. `useId` serve esclusivamente ad associare attributi di accessibilità: `htmlFor`, `aria-describedby`, `aria-labelledby`.

Per più campi nello stesso componente, si usa un prefisso unico:

```
const id = useId();
```

```
<label htmlFor={`${id}-nome`} >Nome</label>
```

```
<input id={`${id}-nome`} />
```

```
<label htmlFor={`${id}-email`} >Email</label>
```

```
<input id={`${id}-email`} />
```

Suspense Component & Pattern

Definizione. Suspense è un componente che permette di mostrare un contenuto di attesa (*fallback*) mentre qualcosa non è ancora pronto.

Uso principale: il code splitting con lazy.

```
import { lazy, Suspense } from 'react';
```

```
const Dashboard = lazy(() => import('./screens/Dashboard'));
```

```
const Impostazioni = lazy(() => import('./screens/Impostazioni'));
```

```
const App = () => (
```

```
  <Router>
```

```
    <Navbar />
```

```
    <Suspense fallback={ <h2>Caricamento pagina...</h2> }>
```

```
      <Routes>
```

```
        <Route path="/dashboard" element={ <Dashboard /> } />
```

```
        <Route path="/impostazioni" element={ <Impostazioni /> } />
```

```
      </Routes>
```

```
    </Suspense>
```

```
  </Router>
```

```
);
```

Cosa cambia davvero. Senza `lazy`, `npm run build` produce un unico file JavaScript con tutta l'applicazione: l'utente che apre la home scarica anche il codice della dashboard, delle impostazioni e di ogni altra pagina. Con `lazy`, il bundle viene spezzato in pezzi e ogni pagina viene scaricata solo quando serve. Su applicazioni grandi la differenza sul primo caricamento è sostanziale.

Suspense copre l'attesa dello scaricamento con il fallback.

Puoi usare più Suspense a granularità diversa, ed è spesso la scelta migliore:

```
<Suspense fallback={<SkeletonHeader />}>  
  <Header />  
</Suspense>  
<Suspense fallback={<SkeletonLista />}>  
  <ListaProdotti />  
</Suspense>
```

Ogni sezione appare appena è pronta, invece di aspettare la più lenta.

Nota sullo stato attuale. Suspense per il data fetching (non solo per `lazy`) è diventato pienamente utilizzabile con React 19 e con i framework come Next.js. Il corso lo presenta principalmente nella forma `lazy + Suspense`, che è quella stabile e immediatamente applicabile.